



以CNN模型 分析FER-2013資料集

深度學習-第四組

組員1 - 資管四乙11044211 賴冠臻

組員2 - 資管四乙11044261 林佩臻

組員3 - 資管四乙11044262 潘葉依琳

組員4 - 資管四乙11044271 張瑋鐘

目錄

01. 資料集介紹

02. 模型介紹

03. 程式碼介紹

04. 結論分析



01.

資料集介紹

資管四乙 11044261 林佩臻

FER-2013資料集

Kaggle平台
網址:<https://www.kaggle.com/datasets/msmbare/fer2013>



0=憤怒(Anger)



1=厭惡Disgust



2=恐懼(Fear)



3=快樂(Happy)



4=悲傷Sad



5=驚訝(Surprise)



6=中性(Natural)

pixels:圖像

- 48 x 48像素灰階人臉照片

Usage:資料種類

- 訓練集 (Training) : 28709
- 公共測試集 (Public Test) : 3589
- 私有測試集 (Private Test) : 3589

emotion:表情對應數字

- 由七種情緒組成
- 分別對對應數字0～6

	emotion		pixels	usage
0	emotion		pixels	Usage
1	0	70 80 82 72 58 58 60 63 54 58 60 48 89 115 121...		Training
2	0	151 150 147 155 148 133 111 140 170 174 182 15...		Training
3	2	231 212 156 164 174 138 161 173 182 200 106 38...		Training
4	4	24 32 36 30 32 23 19 20 30 41 21 22 32 34 21 1...		Training

02.

模型介紹

資管四乙 11044262 潘葉依琳

避免虛假關係:One-hot、Categorical Cross Entropy

<CODE/>

ONE-HOT 編碼 (共 7 類情緒)

```
NUM_CLASS = LEN(NP.UNIQUE(Y_TRAIN))
```

```
Y_TRAIN = (NP.ARANGE(NUM_CLASS) == Y_TRAIN[:, NONE]).ASTYPE(NP.FLOAT32)
```

```
Y_VAL = (NP.ARANGE(NUM_CLASS) == Y_VAL[:, NONE]).ASTYPE(NP.FLOAT32)
```

```
Y_TEST = (NP.ARANGE(NUM_CLASS) == Y_TEST[:, NONE]).ASTYPE(NP.FLOAT32)
```

<CODE/>

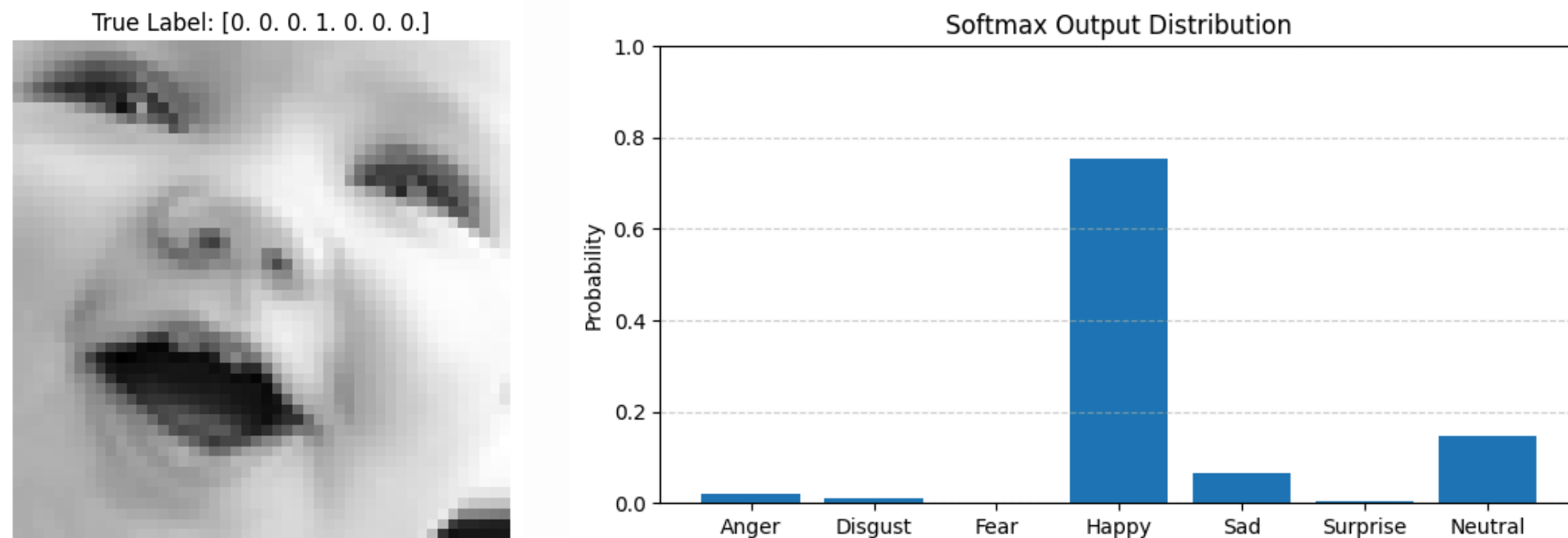
編譯模型

```
MODEL.COMPILE(LOSS='CATEGORICAL_CROSSENTROPY', OPTIMIZER='ADAM', METRICS=['ACCURACY'])
```

為什麼不能直接使用 0~6：整數標籤 vs One-hot 編碼

比較	整數標籤 (0 ~ 6)	ONE-HOT 編碼 + CATEGORICAL CROSS ENTROPY
標籤格式	0,1,2,.....6(整數型)	[0,0,0,1,0,0,0] (one-hot向量)
模型的淺在誤解	誤認情緒間有數值順序或距離關係	各類別獨立，不具順序性
損失計算依據	數值距離(例:3與4差1)	機率分布、one-hot向量間的cross-entropy
可能遇到的問題	Disgust(1)和Fear(2)的差異 比Anger(0)和Surprise(5)小	正確學習分類邊界，不考慮類別間順序

Softmax函式



最後Dense 層輸出是 : logits = [0.2, -1.0, -2.0, 3.5, 0.0, -1.2, 1.0] #這些值只是分數，不能當機率看

經過Softmax函式轉換 : probs = softmax(logits) == [0.02, 0.01, 0.00, 0.75, 0.07, 0.01, 0.14] #加總= 1，代表每一類的機率

$$\text{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)}$$

- Z_i : 第*i*類別的 logit (模型原始輸出分數) ↵
- $\exp(z_i)$: 將分數轉換為正值 凸顯出最有信心的那一類 ↵
- $\sum_{j=1}^K \exp(z_j)$: 對所有類別輸出作 softmax 正規化 ↵
- K : 類別總數 (例如 FER2013 有 7 類) ↵

損失函數 Loss Function : categorical cross entropy

- 比較「模型預測的機率」跟「正確答案的 ONE-HOT 標籤」之間的差距

假設

正確答案是「HAPPY」(第 4 類) $\Rightarrow$ TRUE_LABEL = [0, 0, 0, 1, 0, 0, 0]

模型預測(SOFTMAX) $\Rightarrow$ PROBS = [0.02, 0.01, 0.00, 0.75, 0.07, 0.01, 0.14]

CATEGORICAL CROSS ENTROPY 計算方式：

公式：

$$\text{Loss} = - \sum_{i=1}^C y_i \cdot \log(\hat{y}_i)$$

- C : 類別總數 (例如 FER2013 為 7 類) ↴
- y_i : 真實標籤的第 i 項 (one-hot 表示) ↴
- $\hat{y}_i$: 模型預測的第 i 類別的機率 (softmax 輸出) ↴
- $\log(\hat{y}_i)$: 預測機率的對數，用於衡量信心 ↴

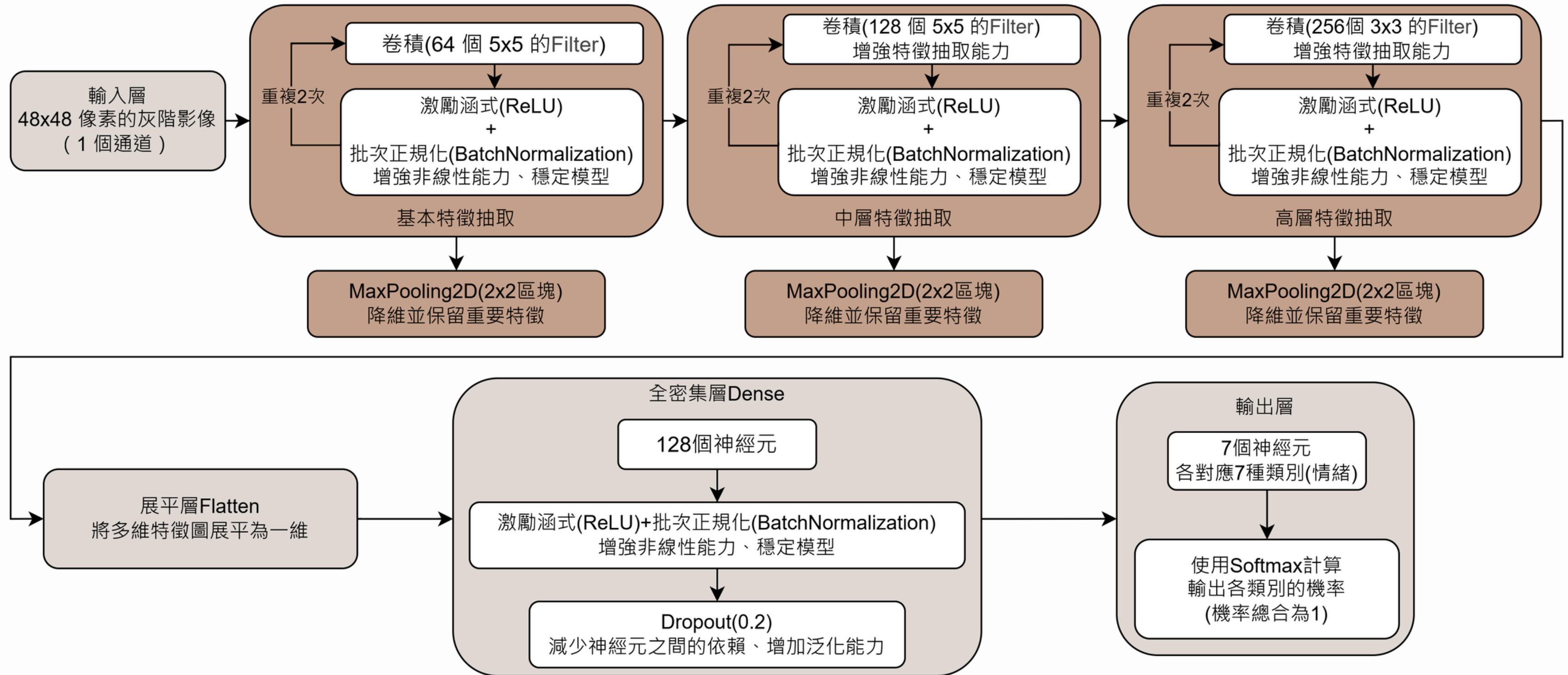
例: 正確答案是「Happy」(第 4 類)，故其它項都乘上 0

$$\text{Loss} = -\log(0.75)$$

$$\equiv \log(0.75) \approx -0.2877$$

$$\equiv \text{Loss} = -(-0.2877) = 0.2877$$

CNN 運作流程



第一版 - 第一組卷積層

卷積 (64 個 5 X 5 的 FILTER)

 基本特徵抽取：提取邊緣、線條、亮暗對比等基礎視覺特徵。

```
input_shape = (48,48,1)
```

```
model.add(Conv2D(64, (5, 5), input_shape=input_shape,activation='relu',  
padding='same'))
```

```
model.add(Conv2D(64, (5, 5), activation='relu', padding='same'))
```

```
model.add(BatchNormalization())
```

```
model.add(MaxPooling2D(pool_size=(2, 2)))
```

第一版 - 第二組卷積層

卷積 (128 個 5X5 的 FILTER): 增強特徵抽取能力

 中層特徵抽取：組合前面的基礎特徵，學習更具結構性的區域 (如眼睛、嘴型輪廓等)

```
model.add(Conv2D(128, (5, 5), activation='relu', padding='same')) # 卷積層
```

```
model.add(Conv2D(128, (5, 5), activation='relu', padding='same')) # 卷積層
```

```
model.add(BatchNormalization())
```

```
model.add(MaxPooling2D(pool_size=(2, 2))) # 最大池化層
```

第一版 - 第三組卷積層

卷積 (256 個 3 X 3 的 FILTER) 增強特徵抽取能力

 高層特徵抽取：辨認出整體臉部表情的形狀變化與情緒關鍵區域（如眉毛上揚、嘴角下垂等）。

```
model.add(Conv2D(256, (3, 3), activation='relu', padding='same')) # 卷積層
model.add(Conv2D(256, (3, 3), activation='relu', padding='same')) # 卷積層
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2, 2))) # 最大池化層
```

第一版 - 展平 + 全密集層 (Flatten + Dense)

 整合所有層級的特徵資訊，進行分類邏輯的判斷與轉換。

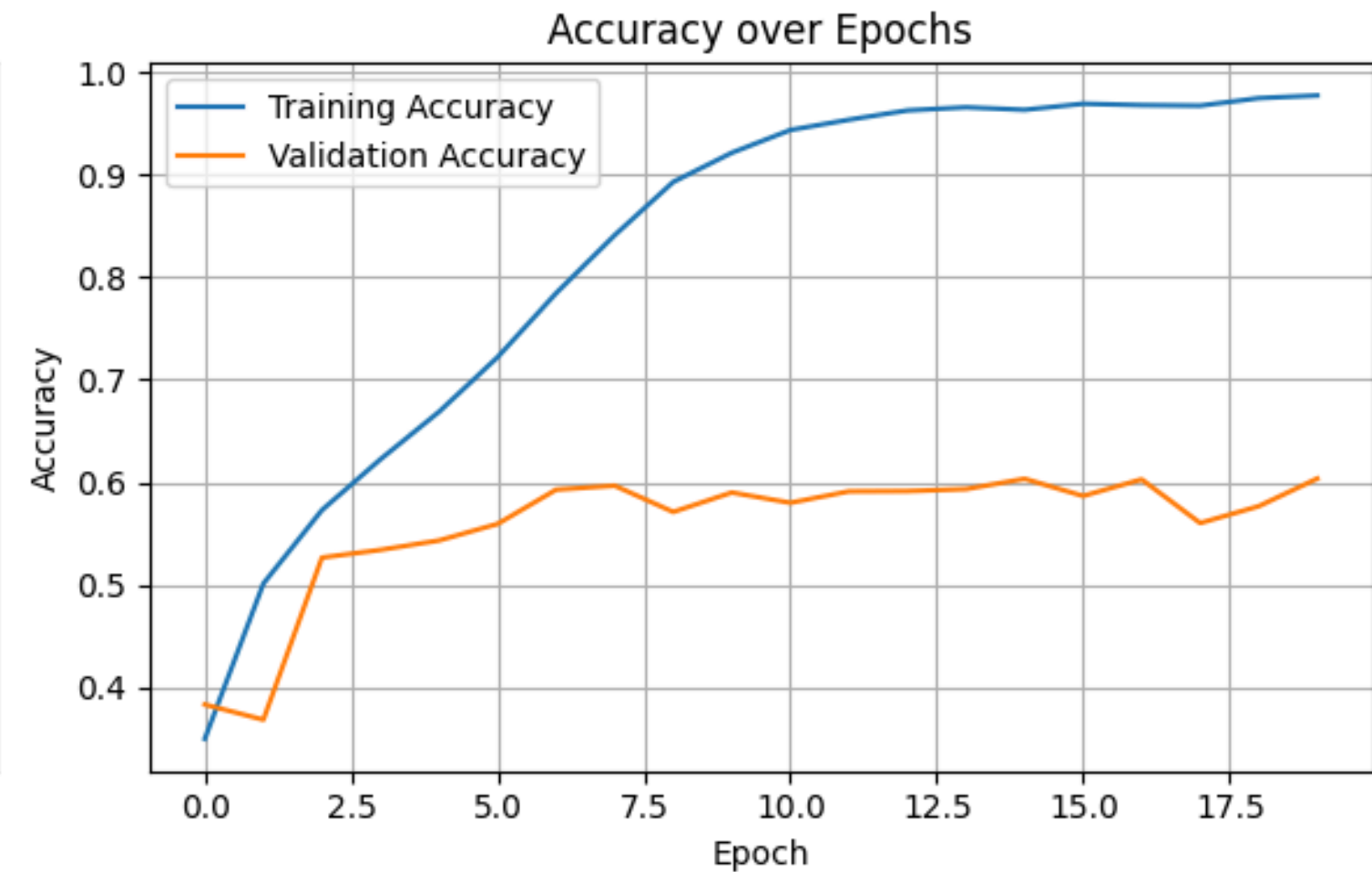
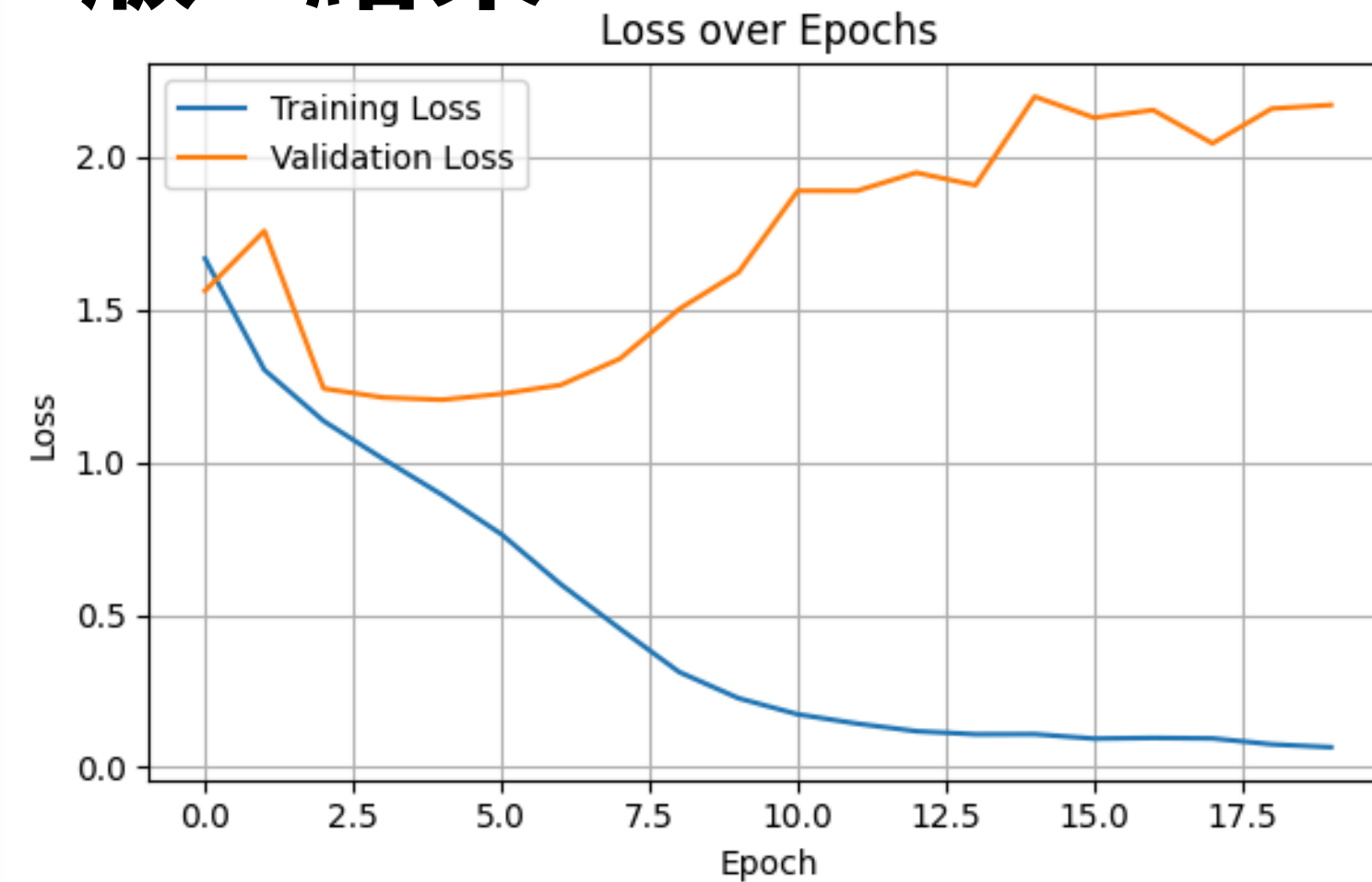
```
model.add(Flatten())  
model.add(Dense(128))  
model.add(BatchNormalization())  
model.add(Activation('relu'))  
model.add(Dropout(0.2)) # 在訓練過程中隨機關閉 20% 的神經元
```

第一版 - 輸出層 + Softmax

 輸出對 7 種情緒的預測機率，找出最可能的表情類別。

```
model.add(Dense(7)) # 7個神經元各對應7種類別(情緒)
model.add(Activation('softmax')) # 使用Softmax計算輸出各類別的機率(機率總合為1)
# 最佳化參數與損失函式設定
# 這裡取用 'categorical_crossentropy' 作為損失函數、採用 Adam 的最佳化方法
model.compile(loss='categorical_crossentropy', metrics=
['accuracy'],optimizer='adam')model.summary()
```

第一版 - 結果



 結果 - 有過擬合的情況

ACCURACY(準確率): 0.9794

LOSS(損失值): 0.0614

VAL_ACCURACY(驗證準確率): 0.6035

VAL_LOSS(驗證損失值): 2.1706

03.

程式碼介紹

資管四乙 11044211 賴冠臻

第二版 - 過擬合優化版: EarlyStopping vs ReduceLROnPlateau

項目	EarlyStopping	ReduceLROnPlateau
功能	停止訓練	降低學習率
解決的問題	避免過度訓練、節省資源	幫助模型繼續學習、突破瓶頸
常搭配使用的情境	監控驗證集表現過擬合	驗證集表現沒進步但模型可能還能變好
是否會停止訓練	是	否（只是改變學習率）

第二版 - 過擬合優化版

 在訓練過程中隨機關閉 50% 的神經元

```
model.add(Dense(128)) # 全連接層
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(0.5))
```

 添加於CNN模型後

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=8,
    restore_best_weights=True,
    verbose=1
)
reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=4,
    verbose=1
)
```

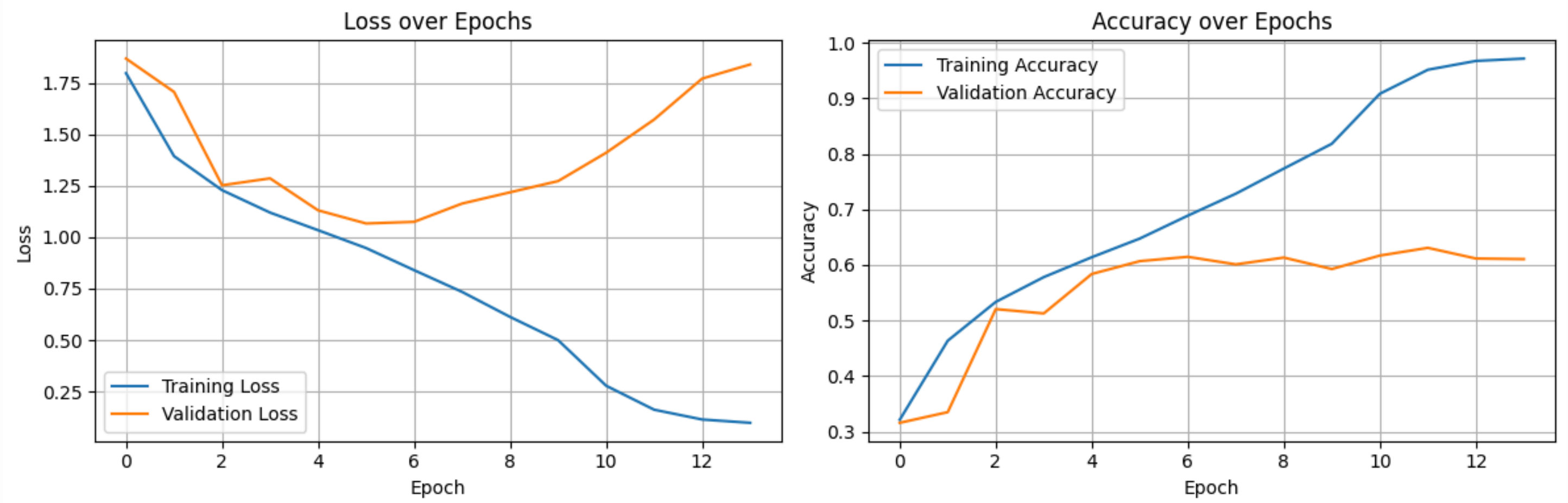
第二版 - 結果

```
Epoch 9/20
449/449 ————— 0s 39ms/step - accuracy: 0.8096 - loss: 0.5293
Epoch 9: ReduceLROnPlateau reducing learning rate to 0.00050000000237487257.
449/449 ————— 18s 41ms/step - accuracy: 0.8095 - loss: 0.5294 - val_accuracy: 0.5823 - val_loss: 1.3826 - learning_rate: 0.0010
Epoch 10/20
449/449 ————— 19s 42ms/step - accuracy: 0.8860 - loss: 0.3496 - val_accuracy: 0.6130 - val_loss: 1.3429 - learning_rate: 5.0000e-04
Epoch 11/20
449/449 ————— 18s 41ms/step - accuracy: 0.9457 - loss: 0.1854 - val_accuracy: 0.6043 - val_loss: 1.5490 - learning_rate: 5.0000e-04
Epoch 12/20
449/449 ————— 19s 42ms/step - accuracy: 0.9679 - loss: 0.1189 - val_accuracy: 0.6094 - val_loss: 1.6125 - learning_rate: 5.0000e-04
Epoch 13/20
449/449 ————— 0s 39ms/step - accuracy: 0.9746 - loss: 0.0944
Epoch 13: ReduceLROnPlateau reducing learning rate to 0.00025000000118743628.
449/449 ————— 20s 41ms/step - accuracy: 0.9746 - loss: 0.0944 - val_accuracy: 0.6024 - val_loss: 1.8378 - learning_rate: 5.0000e-04
Epoch 13: early stopping
Restoring model weights from the end of the best epoch: 5.
總訓練時間: 4.56 分鐘
```

結果 - 學習率

- Epoch 9 : ReduceLROnPlateau 將學習率從 0.001 ➡ 0.0005
- Epoch 13 : 再次降低學習率 ➡ 0.00025
- EarlyStopping 停止訓練，恢復第 5 輪最佳模型

第二版 - 結果



組別	訓練準確率	訓練損失	驗證準確率	驗證損失	分析
第一版	97.90%	0.06	60.30%	2.17	嚴重過擬合
第二版	97.50%	0.09	61.00%	1.83	過擬合稍改善

第三版 - 資料增強



目的：

- 圖片數量有限且角度單一
- 若直接訓練 CNN 模型，容易造成「過擬合」
- 製造出更多樣化的訓練樣本

優點：

- 擴充訓練資料多樣性
- 有助於減少類別間混淆
- 不會改變原始圖片標籤

第三版 - 資料增強

 使用 ImageDataGenerator 對訓練集進行圖像增強，以提高模型泛化能力

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
datagen = ImageDataGenerator(
    rotation_range=30,    # 隨機旋轉角度
    width_shift_range=0.2, # 水平平移
    height_shift_range=0.2, # 垂直平移
    zoom_range=0.2,      # 隨機縮放
    shear_range=0.15,    # 隨機錯切
    horizontal_flip=True  # 隨機左右翻轉
)
train_generator = datagen.flow(
    X_train, y_train,
    batch_size=64
)
```

第三版 - 結果



組別	訓練準確率	訓練損失	驗證準確率	驗證損失	分析
第一版	97.90%	0.06	60.30%	2.17	嚴重過擬合
第二版	97.50%	0.09	61.00%	1.83	過擬合稍改善
第三版	61.20%	1.03	58.10%	1.1	較好泛化，但表現仍待提升

三版F1-score比較

第一版

Classification Report (F1-score):				
	precision	recall	f1-score	support
Anger	0.58	0.46	0.51	491
Disgust	0.72	0.60	0.65	55
Fear	0.48	0.48	0.48	528
Happy	0.75	0.85	0.80	879
Sad	0.46	0.51	0.48	594
Surprise	0.74	0.77	0.76	416
Neutral	0.60	0.50	0.55	626
accuracy			0.61	3589
macro avg	0.62	0.60	0.60	3589
weighted avg	0.61	0.61	0.61	3589

- Fear、Sad 表現較弱
- 所有類別預測分布較不平衡
- Disgust 類表現意外地好（但樣本數較少）

第二版

Classification Report (F1-score):				
	precision	recall	f1-score	support
Anger	0.62	0.44	0.51	491
Disgust	0.55	0.40	0.46	55
Fear	0.49	0.26	0.34	528
Happy	0.73	0.88	0.80	879
Sad	0.41	0.61	0.49	594
Surprise	0.77	0.68	0.72	416
Neutral	0.58	0.55	0.56	626
accuracy			0.60	3589
macro avg	0.59	0.55	0.56	3589
weighted avg	0.60	0.60	0.59	3589

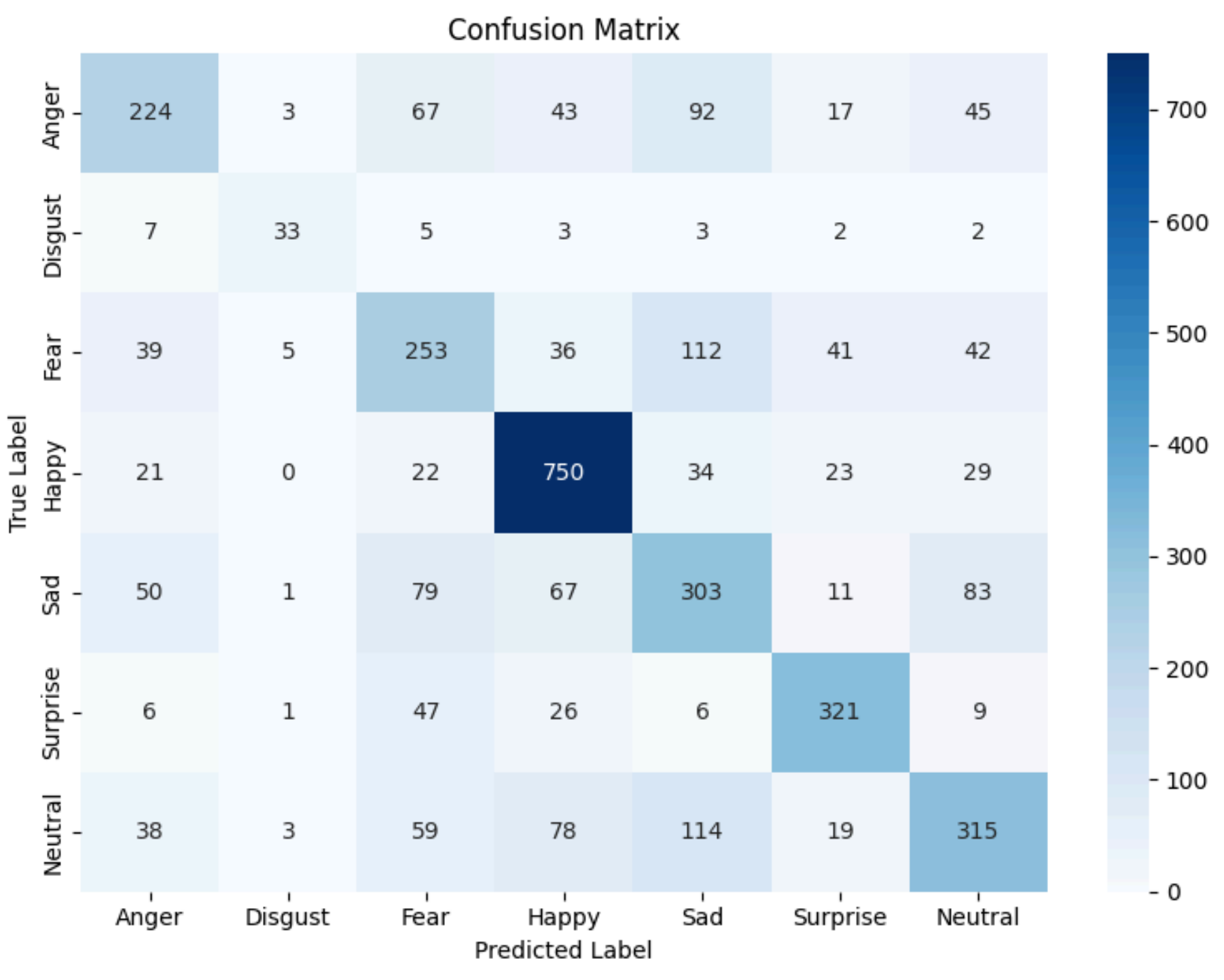
- Fear 與 Disgust 明顯惡化
- 顯示部分正樣本學不到足夠特徵，可能過早停止導致弱類沒學好

第三版

Classification Report (F1-score):				
	precision	recall	f1-score	support
Anger	0.57	0.55	0.56	491
Disgust	0.55	0.20	0.29	55
Fear	0.40	0.43	0.42	528
Happy	0.83	0.87	0.85	879
Sad	0.47	0.50	0.49	594
Surprise	0.68	0.73	0.70	416
Neutral	0.64	0.53	0.58	626
accuracy			0.62	3589
macro avg	0.59	0.54	0.56	3589
weighted avg	0.62	0.62	0.61	3589

- F1-score 在多數類別略為提升
- Disgust 因樣本太少仍不穩定
- 其他類別預測更平均，泛化能力較好

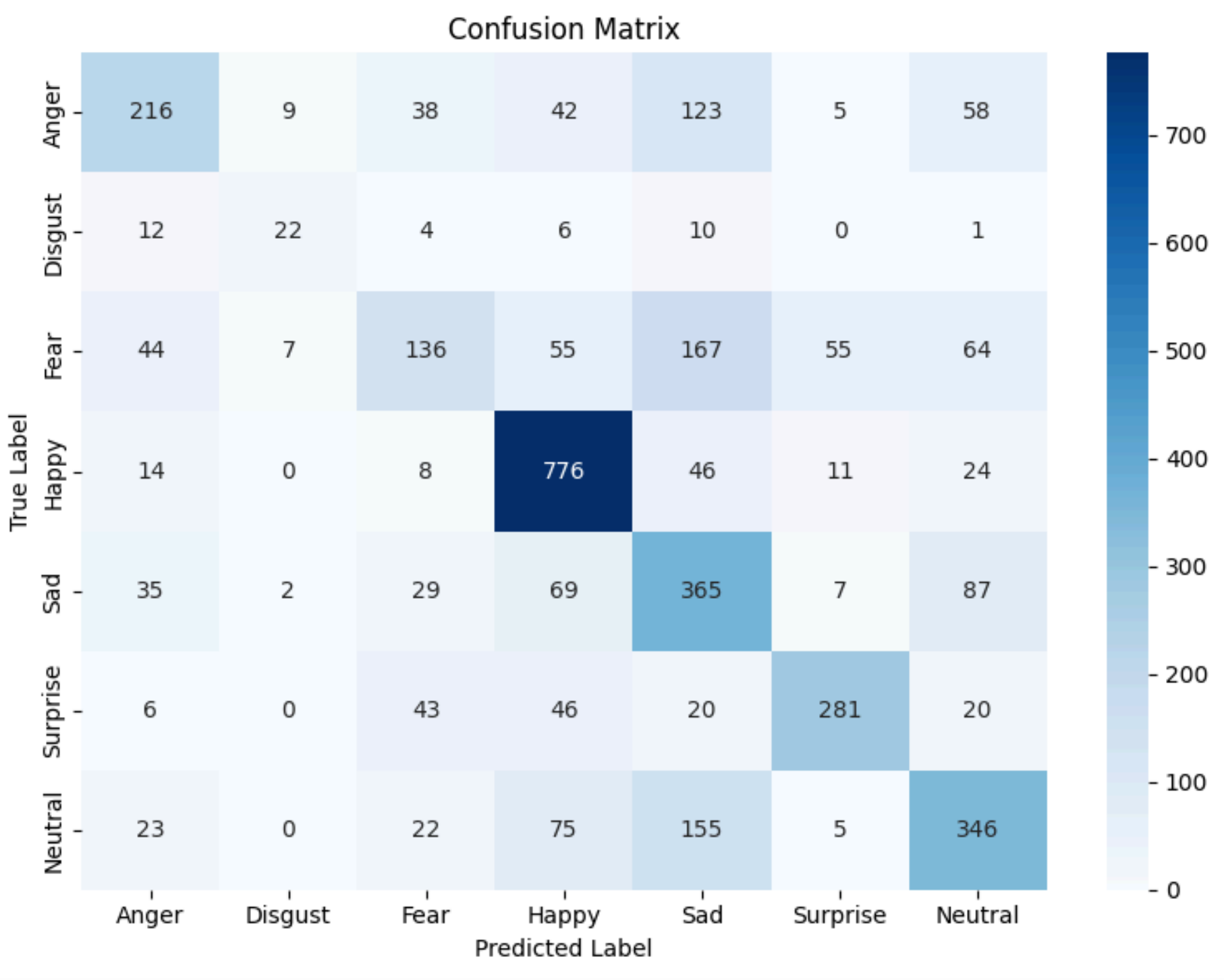
三版混淆矩陣比較



 **第一版**

- Happy 預測表現最佳：750 答對，誤判較少
- Fear、Sad、Neutral 混淆嚴重

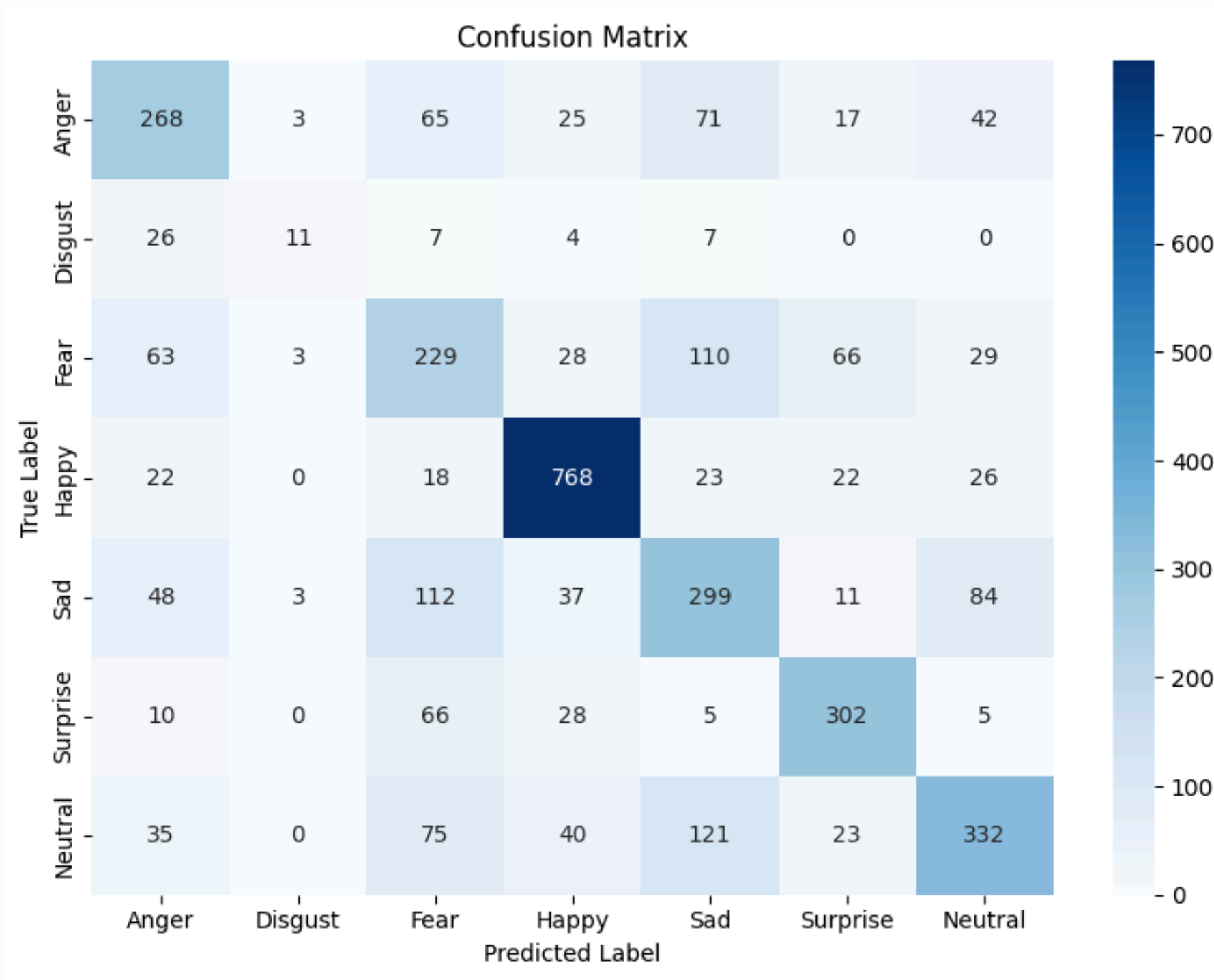
三版混淆矩陣比較



 **第二版**

- Happy答對上升至 776（提升）
- Fear 的正確數下降（253 → 136）
- Sad、Neutral 預測變好
- 特別是 Sad 提升為 365

三版混淆矩陣比較



📌 第三版

- Anger 預測變得非常準 (268)
- Fear 類表現穩定 (229 答對)
- Happy、Neutral、Surprise 三類整體更平衡
- 幾乎所有類別的「非對角線」混淆量明顯下降

04.

結論分析

資管四乙 11044271 張瑋鐘

結論

- 基本CNN架構具備強大特徵抽取能力，但容易產生過擬合。
- 使用EarlyStopping與ReduceLROnPlateau可有效減少不必要訓練輪數。
- Dropout正規化技術在隱藏層中隨機關閉神經元，進一步減緩過擬合現象。
- 資料增強技術，成功提高模型對於不同臉部變化的辨識能力。
- 混淆矩陣顯示第三版模型在多數情緒類別中預測更為平衡與準確。
- 驗證損失從 2.17 降至 1.10，顯示模型泛化能力明顯改善。
- 關鍵發現：資料增強是提升CNN模型泛化能力與類別預測平衡性的核心策略。

分工表

姓名	學號	負責
賴冠臻	11044211	程式、報告
林佩臻	11044261	簡報、報告
潘葉依琳	11044262	程式、報告
張瑋鐘	11044271	簡報、報告

The background of the slide is a collage of numerous small, square, black and white photographs of people's faces, showing a variety of expressions. Two large, light gray, stylized star shapes are positioned on the left side of the slide, one above the other. The word "Thanks!" is written in a large, bold, black, sans-serif font, centered horizontally and partially overlapping the collage and the stars.

Thanks!

THANKS!